

Lesson 01 Demo 01

Installing and Running AutoGen Locally

Objective: To set up AutoGen locally, configure dependencies, and run a simple script that calls the Azure OpenAI model using the AzureOpenAIChatCompletionClient.

Tools required: Python

Prerequisites: OpenAI API Key

Steps to be followed:

1. Open a terminal (Konsole) and set up the environment
2. Create and configure the script file
3. Run the code

Step 1: Open a terminal (Konsole) and set up the environment

1.1 Open the terminal on your system and run the following commands:

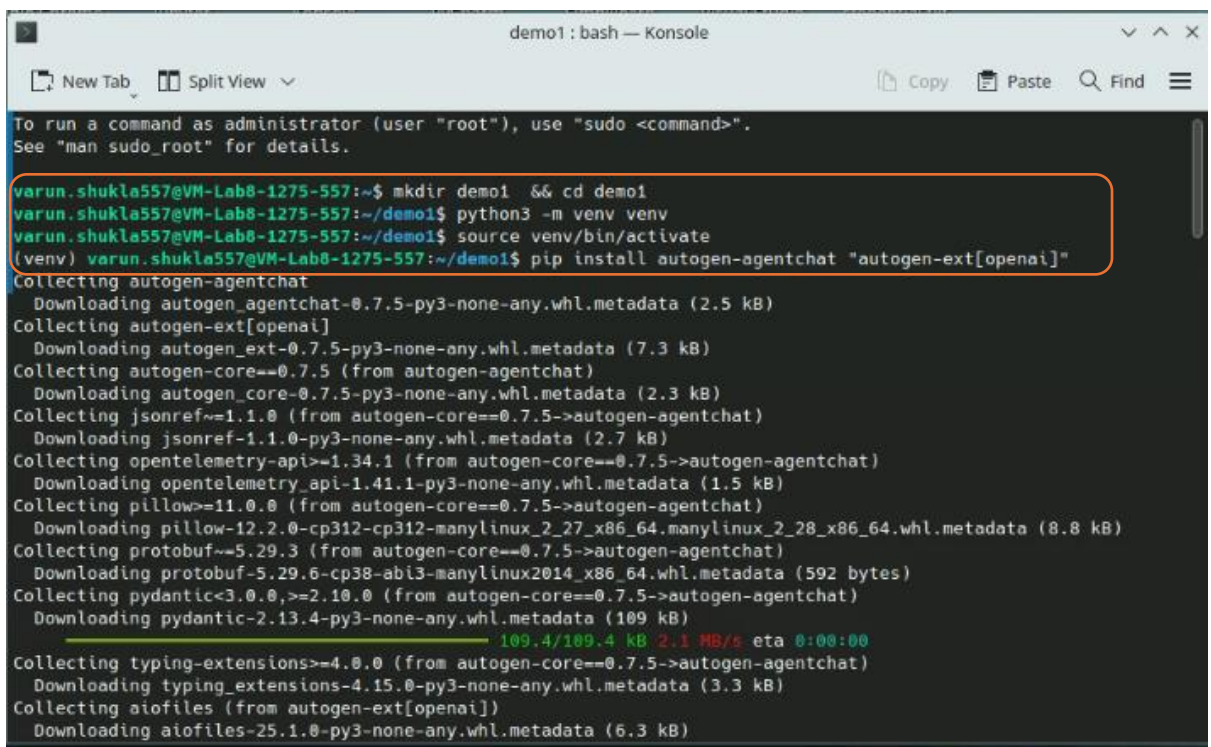
```
mkdir demo1 && cd demo1
```

```
python3 -m venv venv
```

```
source venv/bin/activate
```

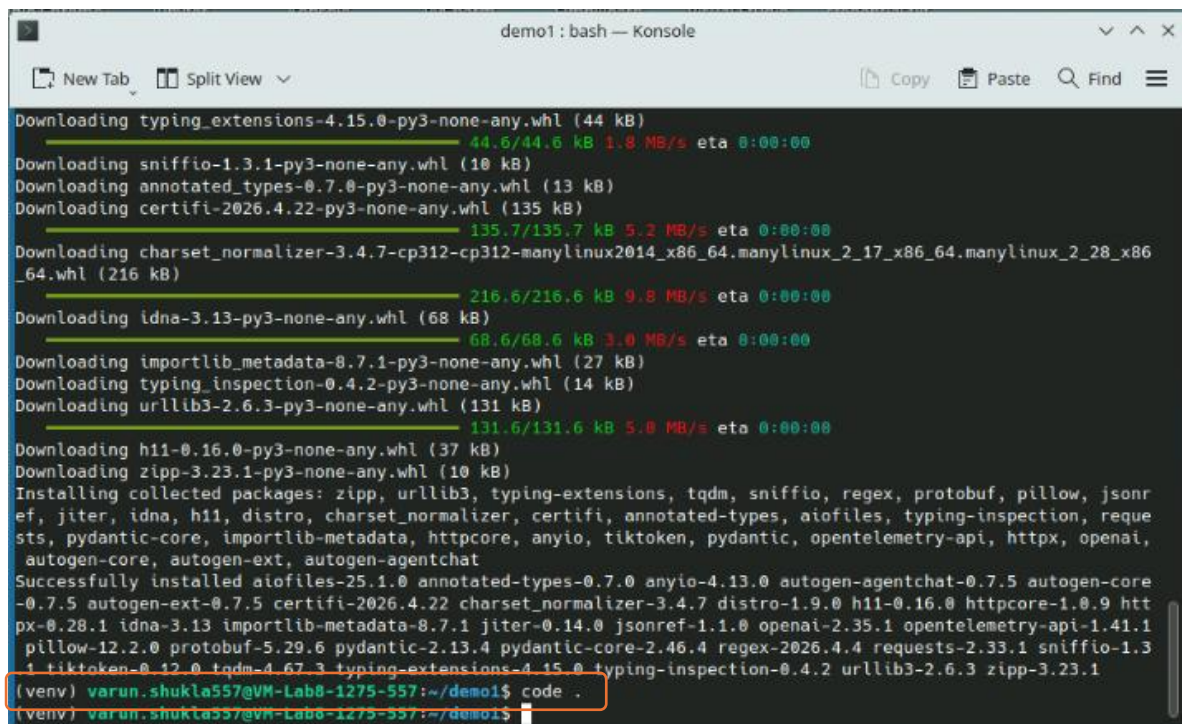
```
pip install autogen-agentchat "autogen-ext[openai]"
```

code .

A screenshot of a terminal window titled "demo1 : bash — Konsole". The terminal shows a series of commands being executed to set up an environment. The commands are: `mkdir demo1 && cd demo1`, `python3 -m venv venv`, `source venv/bin/activate`, and `pip install autogen-agentchat "autogen-ext[openai]"`. The output of the last command shows the installation progress for various packages including autogen-agentchat, autogen-ext[openai], autogen-core, jsonref, opentelemetry-api, pillow, protobuf, pydantic, typing-extensions, and aiofiles. A progress bar for pydantic is visible, showing 109.4/109.4 kB at 2.1 MB/s. The terminal window has a dark background and a light-colored border. The top bar shows the title and some window controls. The bottom of the window shows the command prompt and the current directory.

```
demo1 : bash — Konsole
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

varun.shukla557@VM-Lab8-1275-557:~$ mkdir demo1 && cd demo1
varun.shukla557@VM-Lab8-1275-557:~/demo1$ python3 -m venv venv
varun.shukla557@VM-Lab8-1275-557:~/demo1$ source venv/bin/activate
(venv) varun.shukla557@VM-Lab8-1275-557:~/demo1$ pip install autogen-agentchat "autogen-ext[openai]"
Collecting autogen-agentchat
  Downloading autogen_agentchat-0.7.5-py3-none-any.whl.metadata (2.5 kB)
Collecting autogen-ext[openai]
  Downloading autogen_ext-0.7.5-py3-none-any.whl.metadata (7.3 kB)
Collecting autogen-core==0.7.5 (from autogen-agentchat)
  Downloading autogen_core-0.7.5-py3-none-any.whl.metadata (2.3 kB)
Collecting jsonref~=1.1.0 (from autogen-core==0.7.5->autogen-agentchat)
  Downloading jsonref-1.1.0-py3-none-any.whl.metadata (2.7 kB)
Collecting opentelemetry-api>=1.34.1 (from autogen-core==0.7.5->autogen-agentchat)
  Downloading opentelemetry_api-1.41.1-py3-none-any.whl.metadata (1.5 kB)
Collecting pillow>=11.0.0 (from autogen-core==0.7.5->autogen-agentchat)
  Downloading pillow-12.2.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (8.8 kB)
Collecting protobuf~5.29.3 (from autogen-core==0.7.5->autogen-agentchat)
  Downloading protobuf-5.29.6-cp38-abi3-manylinux2014_x86_64.whl.metadata (592 bytes)
Collecting pydantic<3.0.0,>=2.10.0 (from autogen-core==0.7.5->autogen-agentchat)
  Downloading pydantic-2.13.4-py3-none-any.whl.metadata (109 kB)
  109.4/109.4 kB 2.1 MB/s eta 0:00:00
Collecting typing-extensions>=4.0.0 (from autogen-core==0.7.5->autogen-agentchat)
  Downloading typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)
Collecting aiofiles (from autogen-ext[openai])
  Downloading aiofiles-25.1.0-py3-none-any.whl.metadata (6.3 kB)
```

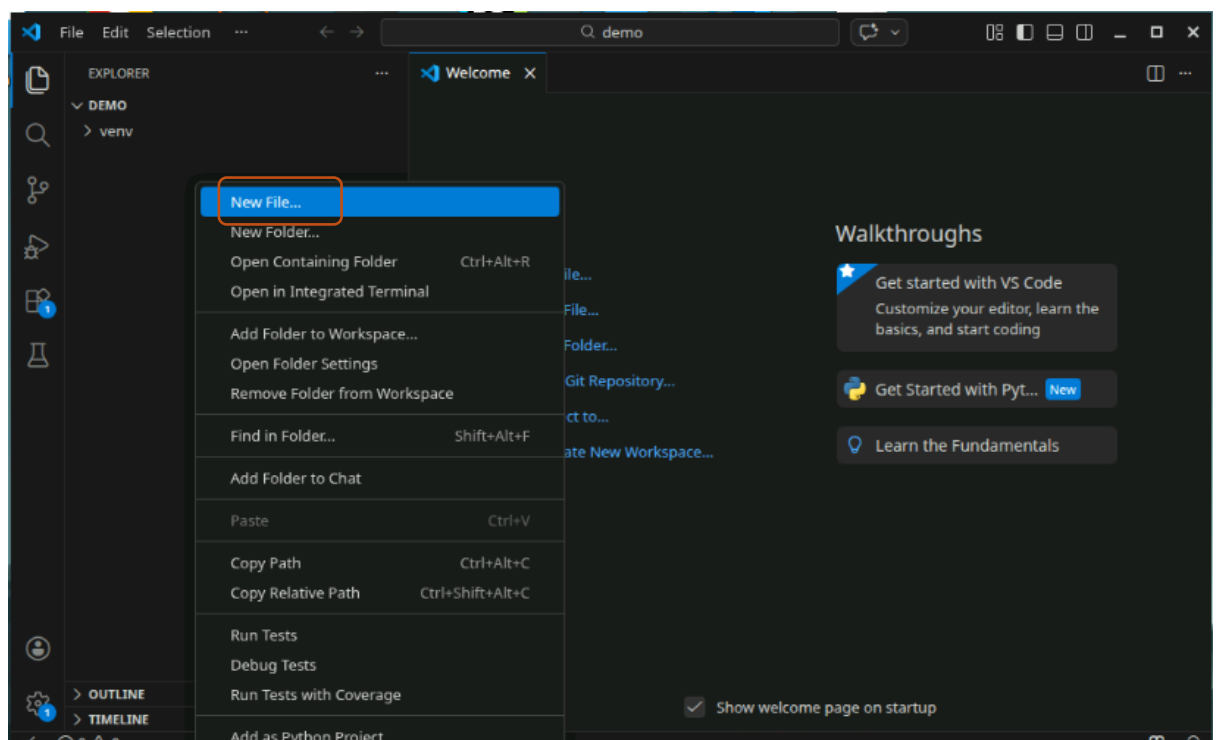


```
demo1: bash — Konsole
New Tab Split View
Copy Paste Find
Downloading typing_extensions-4.15.0-py3-none-any.whl (44 kB)
44.6/44.6 kB 1.8 MB/s eta 0:00:00
Downloading sniffio-1.3.1-py3-none-any.whl (10 kB)
Downloading annotated_types-0.7.0-py3-none-any.whl (13 kB)
Downloading certifi-2026.4.22-py3-none-any.whl (135 kB)
135.7/135.7 kB 5.2 MB/s eta 0:00:00
Downloading charset_normalizer-3.4.7-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (216 kB)
216.6/216.6 kB 9.8 MB/s eta 0:00:00
Downloading idna-3.13-py3-none-any.whl (68 kB)
68.6/68.6 kB 3.0 MB/s eta 0:00:00
Downloading importlib_metadata-8.7.1-py3-none-any.whl (27 kB)
Downloading typing_inspection-0.4.2-py3-none-any.whl (14 kB)
Downloading urllib3-2.6.3-py3-none-any.whl (131 kB)
131.6/131.6 kB 5.0 MB/s eta 0:00:00
Downloading h11-0.16.0-py3-none-any.whl (37 kB)
Downloading zipp-3.23.1-py3-none-any.whl (10 kB)
Installing collected packages: zipp, urllib3, typing_extensions, tqdm, sniffio, regex, protobuf, pillow, jsonref, jiter, idna, h11, distro, charset_normalizer, certifi, annotated_types, aiofiles, typing_inspection, requests, pydantic_core, importlib_metadata, httpcore, anyio, tiktoken, pydantic, opentelemetry_api, httpx, openai, autogen_core, autogen_ext, autogen_agentchat
Successfully installed aiofiles-25.1.0 annotated_types-0.7.0 anyio-4.13.0 autogen_agentchat-0.7.5 autogen_core-0.7.5 autogen_ext-0.7.5 certifi-2026.4.22 charset_normalizer-3.4.7 distro-1.9.0 h11-0.16.0 httpcore-1.0.9 httpx-0.28.1 idna-3.13 importlib_metadata-8.7.1 jiter-0.14.0 jsonref-1.1.0 openai-2.35.1 opentelemetry_api-1.41.1 pillow-12.2.0 protobuf-5.29.6 pydantic-2.13.4 pydantic_core-2.46.4 regex-2026.4.4 requests-2.33.1 sniffio-1.3.1 tiktoken-0.12.0 tqdm-4.67.3 typing_extensions-4.15.0 typing_inspection-0.4.2 urllib3-2.6.3 zipp-3.23.1
(venv) varun.shukla557@VM-Lab8-1275-557:~/demo1$ code .
(venv) varun.shukla557@VM-Lab8-1275-557:~/demo1$
```

Note: This creates a virtual environment inside the **demo** directory and opens the folder in Visual Studio Code.

2. Create and configure the script file agent.py

2.1 Right-click under the Explorer section... to create a new file, and name it agent.py.



2.2 Open the file and paste the following code and save the file:

```
import asyncio
import os
from autogen_core.models import UserMessage
from autogen_ext.models.openai import AzureOpenAIChatCompletionClient

os.environ["AZURE_OPENAI_API_KEY"] =
"2ABecnfxzhRg4M5D6pBKiqxXVhmGB2WvQ0aYKkbTCPsj0JLKsZPfJQQJ99BDAC77b
zfXJ3w3AAABACOGi3sC"

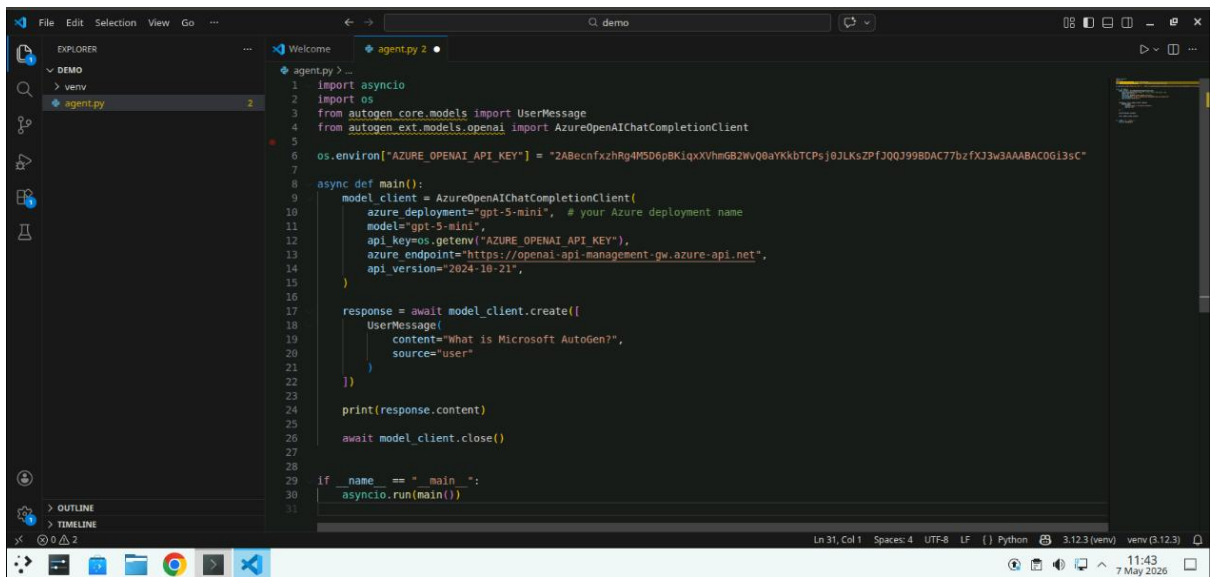
async def main():
    model_client = AzureOpenAIChatCompletionClient(
        azure_deployment="gpt-5-mini", # your Azure deployment name
        model="gpt-5-mini",
        api_key=os.getenv("AZURE_OPENAI_API_KEY"),
        azure_endpoint="https://openai-api-management-gw.azure-api.net",
        api_version="2024-10-21",
    )

    response = await model_client.create([
        UserMessage(
            content="What is Microsoft AutoGen?",
            source="user"
        )
    ])

    print(response.content)

    await model_client.close()

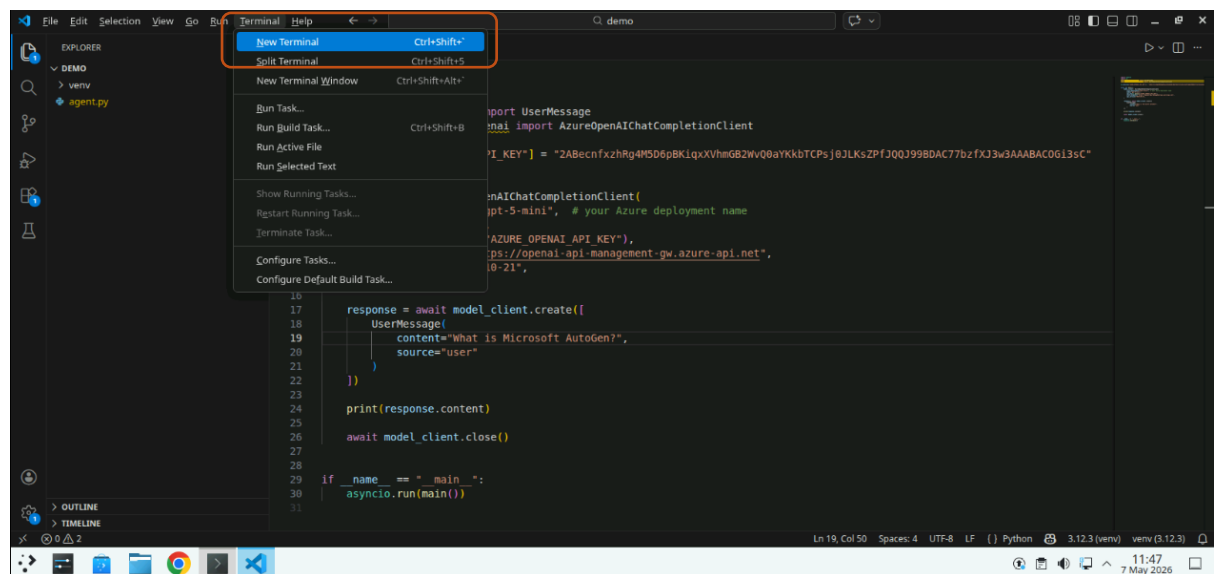
if __name__ == "__main__":
    asyncio.run(main())
```



```
1 import asyncio
2 import os
3 from autogen_core.models import UserMessage
4 from autogen_ext.models.openai import AzureOpenAIChatCompletionClient
5
6 os.environ["AZURE_OPENAI_API_KEY"] = "2ABecnfxzhRg4MSD6pBKlqxVhmG82WvQ0aYKkbTcPsj0JLksZPFJQQJ998DAC77bzfxJ3w3AAABAC0613sc"
7
8 async def main():
9     model_client = AzureOpenAIChatCompletionClient(
10         azure_deployment="gpt-5-mini", # your Azure deployment name
11         model="gpt-5-mini",
12         api_key=os.getenv("AZURE_OPENAI_API_KEY"),
13         azure_endpoint="https://openai-api-management-gw.azure-api.net",
14         api_version="2024-10-21",
15     )
16
17     response = await model_client.create([
18         UserMessage(
19             content="What is Microsoft AutoGen?",
20             source="user"
21         )
22     ])
23
24     print(response.content)
25
26     await model_client.close()
27
28
29 if __name__ == "__main__":
30     asyncio.run(main())
31
```

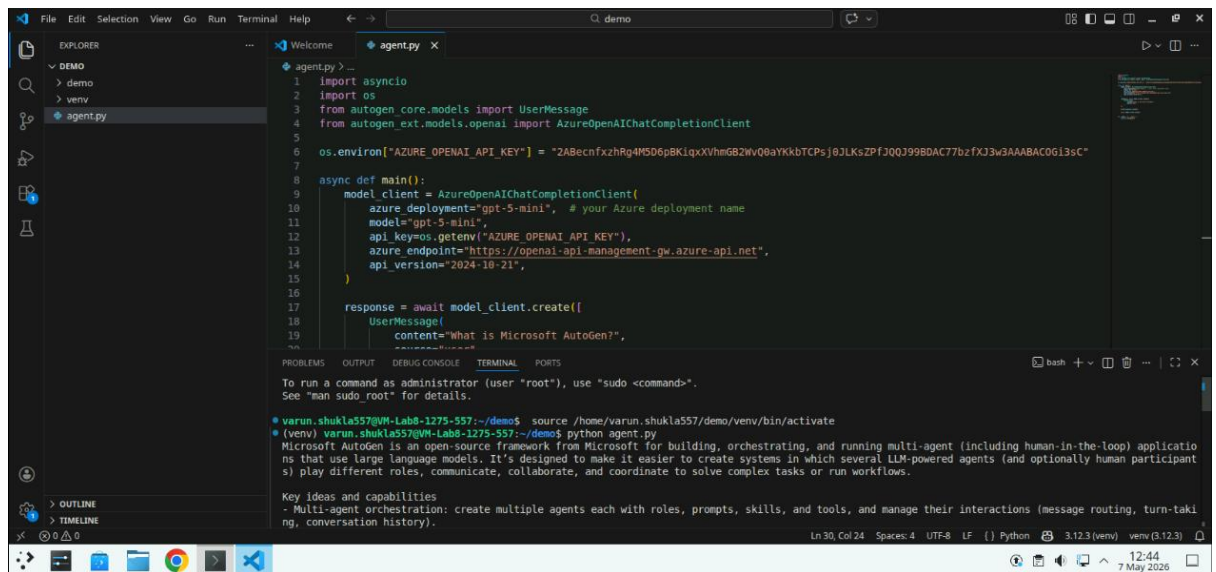
3. Run the code

3.1 Now, open a new terminal by clicking on the terminal



3.2 Now type this command to run the script:

python agent.py



The screenshot shows a Visual Studio Code editor window with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'demo' with subfolders 'demo' and 'venv', and a file 'agent.py'. The 'agent.py' file is open in the editor, showing the following code:

```
1 import asyncio
2 import os
3 from autogen_core.models import UserMessage
4 from autogen_ext.models.openai import AzureOpenAIChatCompletionClient
5
6 os.environ["AZURE_OPENAI_API_KEY"] = "2ABecnfXzhRg4MSD6pBKlqxVhm6B2wvQ0aYKkbTCpsj0JLKsZPf3QQJ99BDAC77bzfx33w3AAABAC0G13sC"
7
8 async def main():
9     model_client = AzureOpenAIChatCompletionClient(
10         azure_deployment="gpt-5-mini", # your Azure deployment name
11         model="gpt-5-mini",
12         api_key=os.getenv("AZURE_OPENAI_API_KEY"),
13         azure_endpoint="https://openai-api-management-gw.azure-api.net",
14         api_version="2024-10-21",
15     )
16
17     response = await model_client.create(
18         UserMessage(
19             content="What is Microsoft AutoGen?",
20         )
21     )
22
23     print(response)
```

The terminal at the bottom shows the command `python agent.py` being executed. The output is:

```
Microsoft AutoGen is an open-source framework from Microsoft for building, orchestrating, and running multi-agent (including human-in-the-loop) applications that use large language models. It's designed to make it easier to create systems in which several LLM-powered agents (and optionally human participants) play different roles, communicate, collaborate, and coordinate to solve complex tasks or run workflows.
```

Key ideas and capabilities

- Multi-agent orchestration: create multiple agents each with roles, prompts, skills, and tools, and manage their interactions (message routing, turn-taking, conversation history).

By following these steps, you have successfully set up AutoGen locally with Azure OpenAI. You created a project, installed dependencies, added your API key, and ran a simple script to confirm everything works.